

The Air-Gapped AI Development Pipeline:

Applying GAMP-Inspired Validation Principles
to Staged Agent Software Development

Stuart J. Hunt

Independent Researcher; Automation Engineer & Process Control Specialist

Correspondence: stuartj1-1981 · GitHub: github.com/stuartj1-1981/Quantum-Swarm

Preprint deposited March 2026 · doi:10.5281/zenodo.XXXXXXXX · CC BY 4.0

Abstract

AI coding agents are increasingly capable of autonomous software development, yet the workflows governing their use remain largely informal. Most teams either allow agents to commit directly to repositories or rely on a single human review of the resulting diff. This paper presents an alternative: a four-stage pipeline that enforces structural separation between development, assessment, deployment, and validation. Inspired by GAMP (Good Automated Manufacturing Practice) validation principles used in pharmaceutical and process automation, the pipeline introduces an air-gapped assessment stage where the reviewing agent has no access to the codebase, and a context-free validation stage that mechanically verifies the deployed implementation against the controlled instruction document. The approach has been in continuous production use since January 2026 on a real-time control system (Quantum Swarm Heating, doi:10.5281/zenodo.18903154). This paper describes the methodology, its rationale, and the practical outcomes observed.

Keywords: *AI-assisted development; software governance; GAMP validation; air-gapped review; staged agent pipeline; instruction document; change control; deployment verification*

1. Introduction

The adoption of AI agents in software development has accelerated dramatically. Tools such as Claude Code, GitHub Copilot Workspace, and Cursor provide agents that can write, test, and commit code with minimal human intervention. However, the governance frameworks around these tools have not kept pace with their capabilities.

In regulated industries, independent review of specifications and procedures is fundamental. Pharmaceutical manufacturing (GAMP 5), process automation (ISA-88/95), and safety-critical systems (IEC 61508) all mandate structural separation between specification, review, and execution. These frameworks exist because experience has shown that combining these roles — or allowing the same context to persist across them — leads to systematic blind spots, regardless of the competence of the individual involved.

The same principle applies to AI agents. An agent that develops code has built up context, assumptions, and momentum during the development process. Asking that same agent (or the same session) to objectively assess its own output introduces the same bias that validation frameworks are designed to prevent. The air-gapped pipeline described in this paper applies this insight to AI-assisted development.

2. The Problem with Current AI Development Workflows

AI-assisted development workflows span a broad spectrum, from fully autonomous commit-and-push agents to highly structured multi-stage pipelines. In practice, most teams operate at one of two points on this spectrum:

Direct commit: The agent writes code and pushes it to the repository, sometimes creating a pull request. The human reviews the diff after the fact. This is fast but provides no structural guarantee that the changes were assessed before reaching the codebase.

Same-session review: The human asks the agent to review its own work within the same conversation. The agent has full context of its development decisions, making it structurally incapable of providing an independent assessment. This is analogous to a developer reviewing their own merge request.

Intermediate approaches exist. Some teams use PR-based review by a different human reviewer, and some agent frameworks separate planning from execution (e.g., distinct planning and coding agents). These improve on the two extremes but typically lack two properties that regulated industries consider essential: first, a controlled specification document that captures intent, reasoning, and evidence independently of the code; and second, a structural guarantee that the deployed implementation faithfully reproduces that specification. The pipeline described here addresses both gaps.

3. The Four-Stage Air-Gapped Pipeline

The pipeline comprises four stages, each executed by a different agent instance with different access permissions and, critically, different context. The human acts as the gatekeeper between stages.

3.1 Stage 1: Develop (Cowork)

The development agent operates in a sandboxed environment with access to a local copy of the codebase. It writes code, runs tests, and iterates until the implementation meets the requirements. The environment is configured so that this agent has no ability to push to the remote repository or create pull requests. This is an access control constraint imposed by the pipeline design, not an inherent limitation of the tooling — the same tool could be configured with full repository access, but the pipeline deliberately withholds it.

The primary output of this stage is not the code itself but an **instruction document**. This document describes what was changed, why it was changed, the technical reasoning behind design decisions, calibration data or test results that validate the approach, and the exact steps required to reproduce the changes. The instruction document is sequentially numbered (INSTRUCTION-01, INSTRUCTION-02, etc.) and stored in a dedicated directory within the repository.

The instruction document serves the same purpose as a User Requirements Specification (URS) and Functional Specification (FS) in GAMP terminology. It captures intent and expected behaviour independently of the implementation.

3.2 Stage 2: Assess (Air-Gapped Chat)

The instruction document is reviewed in a separate AI session that has **no access to the repository**. This is the air gap. The assessing agent cannot see the code, cannot run tests, and cannot verify claims against the actual implementation. It can only evaluate the instruction on its own merit: technical soundness of the reasoning, completeness of the specification, consistency with stated constraints, and clarity of the reproduction steps.

This constraint is deliberate. By removing access to the codebase, the assessment is forced to evaluate the *thinking* rather than the *artefact*. An instruction that relies on implicit context will fail this review. Only instructions that are self-contained and technically rigorous will pass.

The human makes the final go/no-go decision based on the assessment. This mirrors the Installation Qualification (IQ) review in GAMP, where an independent party verifies that the specification is adequate before execution begins.

3.3 Stage 3: Deploy (Claude Code)

The approved instruction document is passed to a deployment agent (Claude Code) that has full access to the repository. This agent creates a feature branch and applies the changes described in the instruction. The deployment agent is constrained to implement only what the instruction specifies. It does not improvise, extend scope, or make architectural decisions.

At the end of this stage, the branch exists with the implementation committed but no pull request has been created. The branch and its diff are the inputs to Stage 4.

3.4 Stage 4: Validate (Context-Free Verification)

A fourth agent receives exactly two inputs: the approved instruction document and the branch diff produced by Stage 3. It has no conversation history, no repository context beyond the diff, and no knowledge of why the change was made. Its task is purely mechanical: does this diff implement this specification, fully and exclusively?

This mirrors how a QA reviewer operates in a GxP context. The reviewer does not need the design rationale — that was assessed and approved upstream. They are verifying execution fidelity against a controlled document. If the reviewer needs context to understand the diff, that is itself a finding: it means the instruction document was not self-contained.

The validator maps diff hunks to specification items and flags orphans in either direction: specification items with no corresponding diff (incomplete implementation) or diff hunks with no corresponding specification item (scope creep). The deviation criterion is behavioural: any executable functionality in the diff that is not described in the instruction document constitutes a deviation. Cosmetic changes (formatting, import reordering, whitespace) do not.

If the validator passes, the pull request is created. If it rejects, the rejection report goes to the human. The human decides whether the deviation reflects an implementation error (route back to Stage 3 with corrected

instruction) or an underspecified instruction (route back to Stage 1 for rework). The validator is stateless and has no authority to remediate. It reports. The human decides.

This maps to the Operational Qualification (OQ) stage in GAMP, where the installed system is tested against the approved specification to verify it performs as intended.

4. Pipeline Structure

Table 1 summarises the forward flow of the pipeline. Each stage is executed by a different agent instance with different context and access permissions. The instruction document is the sole artefact that crosses the air gap. The branch diff and instruction document together are the inputs to Stage 4.

Stage	Agent	Access	GAMP Equivalent	Primary Function
1. Develop	Cowork	Sandboxed, no repo push	URS + FS + Design	Write code, run tests, produce instruction document
2. Assess	Chat	Air-gapped, no repo access	Design Review / IQ	Independent evaluation of specification completeness and technical merit
3. Deploy	Code	Full repo access	Execution	Execute specification against system, create feature branch
4. Validate	Code	Diff + instruction only	OQ	Context-free verification that diff matches specification exactly

Table 1: The four-stage air-gapped pipeline with GAMP 5 V-model equivalents. Each stage uses a different agent instance with different context and access permissions.

Pipeline Artefact	GAMP Equivalent	Function
Instruction Document	Batch Record	Controlled record documenting intent, reasoning, evidence, and steps
Human Go/No-Go	QP Release	Authorised person approves transition between stages

Table 2: Pipeline artefacts and their GAMP equivalents.

4.1 The Iterative Review Cycle

The forward flow shown in Table 1 represents the happy path. In practice, the pipeline includes an iterative feedback loop between the Chat (Assess) and Cowork (Develop) stages, mediated by the human at every transition.

When the air-gapped assessor identifies a problem in the instruction document — a gap in reasoning, an unsupported assumption, an incomplete specification — it raises the issue as a comment. The assessor cannot fix the problem itself because it has no access to the codebase. It can only describe what is wrong and why.

The human reads the assessment, decides whether the feedback is valid, and relays it back to the development agent in Cowork. This is a deliberate human-in-the-loop step: the human acts as the interpreter between the two agents, filtering feedback and adding context that the air-gapped assessor may lack.

The development agent revises both the code and the instruction document, then resubmits the updated instruction for a fresh assessment. The assessor evaluates the revision independently — it has no memory of the previous review cycle. This ensures each assessment is unbiased by prior exchanges. The cycle repeats until the human is satisfied that the instruction is complete and the assessment raises no further issues.

4.2 The Validation Failure Loop

A similar feedback path exists at Stage 4. If the validator rejects the implementation, the rejection report returns to the human. The human has two return paths: back to Stage 3 if the instruction was adequate but the deployment agent drifted, or back to Stage 1 if the instruction itself was underspecified. There is no automated retry loop between the deployment agent and the validator. This is deliberate: an automated loop could iterate the implementation away from the approved specification without human oversight.

5. The Instruction Document as a Controlled Record

The instruction document is the central artefact of the pipeline. Unlike a commit message or PR description, it is a standalone specification that captures:

Intent: What problem is being solved and why this approach was chosen.

Reasoning: Technical justification for design decisions, including alternatives considered and rejected.

Calibration data: Empirical evidence supporting parameter choices, with sources and methodology.

Reproduction steps: Deterministic instructions that another agent can execute against the codebase to produce the exact changes.

Test criteria: Expected outcomes that the deployment agent must verify.

This creates a paper trail that is independent of the code history. A commit message records what changed; the instruction document records *why* it changed, what evidence supported the decision, and what the expected outcome was. This is directly analogous to the batch record in pharmaceutical manufacturing, which documents not just the process steps but the reasoning and measurements at each stage.

The addition of Stage 4 (Validate) elevates the instruction document from a communication artefact to a **validated controlled record** in the change control sense. Because the validator performs a mechanical comparison of the diff against the instruction, the document must be precise enough to support deterministic verification. Vague specifications cannot survive this stage. The instruction must enumerate files affected, functions modified, behavioural intent per change, and acceptance criteria. This precision requirement is not overhead — it is the mechanism by which the pipeline enforces specification quality.

6. Mapping to GAMP Validation Principles

Table 1 maps the pipeline stages to the GAMP 5 V-model lifecycle and Table 2 maps the pipeline artefacts to their GAMP equivalents. The correspondence is structural, not superficial. Each pipeline stage performs the same function as its GAMP counterpart.

The GAMP V-model includes a Performance Qualification (PQ) stage beyond OQ, in which the system is verified to perform as intended under real operating conditions over an extended period. The pipeline as described does not include a PQ equivalent. PQ maps to ongoing production monitoring and performance trending — activities that occur after the change has been deployed and the pull request merged. These are operational concerns rather than deployment governance concerns, and are outside the scope of the pipeline. A future extension could incorporate automated regression monitoring as a PQ analogue, but the present work is limited to the IQ–OQ portion of the qualification lifecycle.

7. Why the Air Gap Matters

The air gap between Stage 1 (Develop) and Stage 2 (Assess) is the defining feature of the pipeline. It provides four properties that cannot be achieved by reviewing code within the same agent session.

7.1 Context Independence

The development agent accumulates context, assumptions, and problem-solving momentum over the course of a session. This context is valuable during development but becomes a liability during review. The agent is more likely to rationalise its own decisions than to challenge them. The air gap eliminates this by ensuring the assessor has no shared context with the developer.

7.2 Document Completeness Forcing

When the instruction document is the only artefact that crosses the air gap, it must be self-contained. Any reasoning that exists only in the development session's context window is lost. This forces the development agent to externalise all assumptions, decisions, and evidence into the document. The result is a higher-quality specification than would be produced if the assessor could simply inspect the code.

7.3 Repeatability

A well-written instruction document is deterministic. Any deployment agent can execute it against any branch state and produce the same result. This enables re-execution after merge conflicts, cherry-picking across branches, or replaying instructions on a different codebase version. The instruction document becomes a reusable specification, not a one-time conversation.

7.4 Auditability

The instruction documents form a complete audit trail of every change made to the system. Each document records not just what changed but why, what evidence supported the decision, what alternatives were considered, and what the expected outcomes were. This is qualitatively different from a git log, which records only the what.

8. The Validation Stage: Closing the Specification-Implementation Gap

The original three-stage pipeline ensured that the specification (instruction document) was independently reviewed before deployment. However, it left a gap: there was no structural guarantee that the deployed code faithfully reproduced the specification. The deployment agent was trusted to implement exactly what was specified, but this trust was not verified by an independent party.

In process automation terms, this is the difference between having an IQ (the specification is adequate) and having both an IQ and an OQ (the installed system matches the specification). Stage 4 closes this gap.

8.1 Isolation Model

The validation agent's isolation is more restrictive than the assessment agent's. The assessor at Stage 2 receives only the instruction document but can reason about it in the context of general engineering knowledge. The validator at Stage 4 receives the instruction document and the branch diff, but has no repository context, no conversation history, and no knowledge of the development rationale. This is deliberate: the validator is performing a mechanical comparison, not an engineering review.

8.2 Deviation Criteria

A deviation is any behavioural change to code paths not addressed by the instruction document. This is a behavioural definition, not a textual one. Cosmetic changes (formatting, import reordering, whitespace

normalisation) do not constitute deviations.

The criterion distinguishes between two cases. First, changes to functions or modules that the instruction document does not mention at all: these are deviations regardless of their nature (even if beneficial). Second, defensive additions to code paths that the instruction *does* specify (e.g., null checks, error handling guards): these are at the human's discretion when reviewing the validator's report. The validator flags them; the human decides whether they constitute scope creep or reasonable engineering practice.

This criterion mirrors how deviation is handled in traditional change control: the question is not 'does the output look exactly like the input?' but 'does the output do anything the specification did not authorise?' The distinction matters because deployment agents routinely apply minor formatting changes, while a single undocumented behavioural change could introduce a defect.

8.3 Failure Routing

If the validator rejects the implementation, the rejection report returns to the human. The human determines the root cause and routes accordingly:

Implementation drift: The instruction was adequate but the deployment agent deviated from it. The human routes back to Stage 3 with the existing instruction, potentially with additional guidance to the deployment agent.

Underspecified instruction: The instruction was too vague for the deployment agent to implement precisely. The human routes back to Stage 1 for revision, and the revised instruction goes through Stage 2 (Assess) again before re-entering Stage 3.

There is no automated retry loop between the deployment and validation stages. This is a fundamental design constraint. An automated loop could iterate the implementation away from the approved specification without human oversight, which would defeat the purpose of the controlled document as the authoritative record of intent.

9. Pipeline Failure Modes

The pipeline's effectiveness depends on the quality of its participants — both human and AI. Several failure modes have been identified through production use:

Assessor blind spots: The air-gapped assessor evaluates the instruction document using general engineering knowledge but lacks domain-specific context. An instruction based on a flawed premise (e.g., an incorrect assumption about system behaviour) may pass assessment if the flaw requires codebase access to detect. The pipeline mitigates this through the human gatekeeper, who has both domain knowledge and codebase context, but does not eliminate it. The air gap trades domain sensitivity for review independence — a deliberate design choice.

Human rubber-stamping: The pipeline assumes the human exercises genuine judgment at each go/no-go decision. If the human routinely approves without reading the assessment, the air gap provides no value. This is not unique to the pipeline — every approval-gated process in regulated industry faces the same risk — but the pipeline provides no technical mitigation against it. The instruction archive does, however, create an audit trail that makes rubber-stamping detectable after the fact.

Assessor error: The assessor may raise incorrect objections, causing unnecessary rework. Because the human mediates all feedback, incorrect assessments can be filtered. However, a human who lacks the confidence to overrule an AI assessment may propagate errors. This is a training and experience issue rather than a pipeline design issue, but it affects practical outcomes.

Instruction quality plateau: The pipeline's output quality is bounded by the development agent's ability to write precise, self-contained instruction documents. If the agent consistently produces vague specifications,

the review cycle becomes a bottleneck. In practice, instruction quality has improved over time as the development agent's system prompt and CLAUDE.md instructions have been refined to encode the documentation requirements explicitly.

10. Practical Outcomes

The pipeline has been in continuous production use since January 2026 on a real-time control system (Quantum Swarm Heating, doi:10.5281/zenodo.18903154) comprising over 50 test files, 22 real-time controllers, a digital twin simulation engine, a web-based operator interface, and multiple live installations.

As of March 2026, the instruction archive contains 32 instruction documents (28 completed and deployed, 4 in active development). These cover controller implementations, system identification parameter changes, UX features, dependency audits, and architectural refactoring. Of the 28 completed instructions, the naming convention reflects the iterative review cycle: several bear suffixed variants (e.g., INSTRUCTION-09, 09a, 09b, 09c) indicating cases where the original specification was decomposed into smaller, independently deployable units following assessment feedback. The instruction archive is publicly available in the repository's development/ directory.

The following qualitative observations have been made:

Reduced deployment errors: The air-gapped assessment has caught specification gaps, incorrect assumptions, and incomplete reasoning that would not have been identified by a code diff review alone.

Higher documentation quality: Because the instruction document must survive the air gap and the validation stage, development agents produce significantly more detailed and well-reasoned specifications than they would in a direct-commit workflow.

Cleaner git history: Each pull request corresponds to a single, well-defined instruction. There are no 'fix review comments' or 'address feedback' commits. The PR is either correct on first execution or the instruction is revised and re-deployed.

Knowledge preservation: The instruction archive captures engineering decisions that would otherwise exist only in ephemeral chat sessions. New team members (human or AI) can read the instruction history to understand not just the current state of the code but the reasoning behind it.

Human remains in control: The human makes every go/no-go decision. No code reaches the repository without explicit human approval of the instruction that produced it. This is not a rubber stamp; the air-gapped assessment provides the human with an independent technical evaluation to inform their decision.

11. Broader Applicability

The pipeline is not specific to any particular AI agent, programming language, or domain. The core principles are agent-agnostic and apply wherever AI agents are used to modify codebases or configurations:

Infrastructure as Code: Terraform, Ansible, or Kubernetes manifests where incorrect changes can affect production systems.

Safety-critical software: Medical devices, automotive systems, or industrial control where regulatory frameworks already mandate similar separation.

Multi-team development: Where different teams own different parts of the codebase and need to coordinate changes through a controlled process.

Regulated industries: Finance, healthcare, and energy where audit trails and change control are compliance requirements.

The four-stage variant with validation is particularly relevant to GxP-regulated environments (pharmaceutical, medical device, clinical systems) where CSV (Computer System Validation) and data integrity requirements demand that implementation fidelity to specification is independently verified, not assumed.

The pipeline's *tooling* requirements are minimal: the ability to run multiple agent sessions with different access permissions, and a version-controlled location for instruction documents. No specialised tooling is required beyond what is already available in standard AI development environments. However, the *competence* requirements are not trivial. The human gatekeeper must understand both the technical domain and the validation principles well enough to evaluate assessments, route failures correctly, and distinguish genuine deviations from reasonable engineering judgment. The development agent must be capable of writing instruction documents that are precise and self-contained enough to survive the air gap and the validation stage. Teams without experience in structured specification or change control may find the learning curve significant. This is, however, also a strength: the methodology rewards engineering discipline and produces better outcomes as that discipline matures.

12. Limitations and Future Work

The pipeline introduces additional overhead compared to direct-commit workflows. Each change requires writing an instruction document, conducting an air-gapped assessment, deploying through a separate agent, and validating the result. For trivial changes (typo fixes, dependency updates), this overhead may not be justified. The pipeline is most valuable for changes that involve technical reasoning, parameter calibration, or architectural decisions.

The quality of the air-gapped assessment depends on the quality of the instruction document. A vague or incomplete instruction will produce a correspondingly weak assessment and will likely be rejected at the validation stage. The pipeline does not eliminate the need for engineering judgment; it structures how that judgment is applied and documented.

The validation stage's deviation criterion (Section 8.2) is clear in principle but admits edge cases in practice. The distinction between 'deviation from specification' and 'reasonable engineering judgment' is never fully mechanical — a tension that mirrors the same ambiguity in traditional QA. The pipeline does not resolve this ambiguity; it surfaces it explicitly in the validator's report and places the judgment with the human.

Future work includes automated instruction template validation (does the document contain all required sections?), metrics for instruction quality over time, integration with existing change management systems (Jira, ServiceNow) to create traceability from requirements through instructions to deployed code, and formalisation of the deviation rubric for regulated environments.

13. Conclusion

The air-gapped AI development pipeline applies established validation principles from process automation and pharmaceutical manufacturing to the emerging field of AI-assisted software development. By enforcing structural separation between development, assessment, deployment, and validation, it addresses the governance gap in current AI coding workflows.

The key insight is that the instruction document, not the code, is the controlled artefact. Code is an implementation detail. The instruction captures the intent, reasoning, and evidence that produced the code. By making this the unit of review and the medium of communication between pipeline stages, the process ensures that every change to the system is traceable, auditable, and reproducible.

The addition of the validation stage closes the final gap: the assurance that what was deployed is exactly what was specified. In regulated terminology, it completes the qualification lifecycle from IQ through OQ. For environments where this level of assurance is a compliance requirement, the four-stage pipeline provides a

practical, implementable framework using tools that exist today.

The principles described here are not theoretical. They have been developed through continuous production use since January 2026 on a real-time control system and refined iteratively based on practical experience. They are offered to the wider community as a contribution to the developing practice of responsible AI-assisted engineering.

14. Data Availability

The production system on which this pipeline was developed and validated (Quantum Swarm Heating) is available as an open-source repository at github.com/stuartj1-1981/Quantum-Swarm and deposited on Zenodo (doi:10.5281/zenodo.18903154, CC BY 4.0). The instruction document archive demonstrating the pipeline in practice is contained within the repository's development/ directory. No additional data are required to reproduce or evaluate the methodology described in this paper.

15. References

- [1] ISPE. GAMP 5: A Risk-Based Approach to Compliant GxP Computerized Systems. 2nd Edition. ISPE, 2022.
- [2] ISA. ISA-88.01: Batch Control Part 1: Models and Terminology. Instrument Society of America, Research Triangle Park NC, 1995.
- [3] ISA. ISA-95.00.01: Enterprise-Control System Integration Part 1: Models and Terminology. ISA, 2010.
- [4] IEC. IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems. IEC, 2010.
- [5] Hunt, S.J. Physics-Based Thermal Optimisation for Residential Heat Pumps: Fleet Simulation, Weather Compensation Analysis, and the Shoulder Mode Discovery. Zenodo dataset and preprint record, doi:10.5281/zenodo.18903154, 2026.
- [6] Anthropic. Claude Code: AI-assisted software development. <https://claude.ai/claude-code>, 2025.
- [7] GitHub. GitHub Copilot Workspace. <https://github.com/features/copilot>, 2024.
- [8] FDA. 21 CFR Part 11: Electronic Records; Electronic Signatures. U.S. Food and Drug Administration, 2003.
- [9] MHRA. Data Integrity Guidance. Medicines and Healthcare products Regulatory Agency, 2018.

16. AI Contribution Disclosure

AI agents (Anthropic Claude, via Claude Code and Cowork) were used as development tools in the preparation of this manuscript and in the production system it describes. All intellectual content, editorial decisions, methodology design, and assertions of fact are the sole responsibility of the named author. No AI agent is listed as an author. This disclosure is made in accordance with Zenodo's requirements for transparency regarding AI-assisted content creation.

This paper was developed collaboratively using the pipeline it describes. The content was developed in Cowork mode (Stage 1) and the resulting document is available for assessment (Stage 2) and publication (Stage 3).